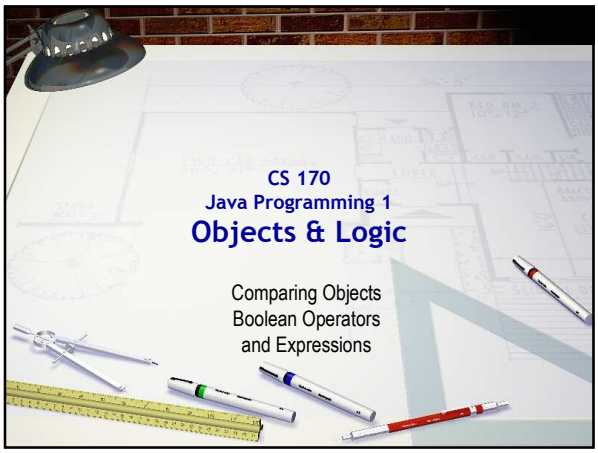


Express Yourself

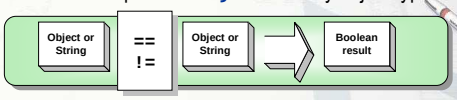
- Open Microsoft Word and create a new document
- Save the file as LastFirst_ic11.doc
  - Replace LastFirst with your actual name
- Put your name and today's date at the top of the sheet
- Title it "CS 170 In-class Exercise 11"
- **Exercise 1**: Download ic11.jar and open using Open Non-BlueJ. Inside your project create a class Fraction. A Fraction has two fields, numerator and denominator, a constructor that takes two ints, and three accessors getNumerator, getDenominator, and toString

A Few Fine Points About if

- Many languages use the **=** symbol for equality
  - **if (x = 10)** won't compile in Java
  - **if (ok = true)** does compile, but is **always** incorrect
- **Never** compare against the literals **true** or **false**
  - **if (ok == true)** is very very poor style
- Many languages use **<>** to mean not-equal
  - **if (x <> 10)** won't compile in Java
- Use **>=** rather than **=>** which doesn't compile

Object Relations

- When comparing objects and **Strings** you can only use the **==** and **!=** operators, not **>**, **<**, **>=**, or **<=**
  - You should not mix different object types
  - Don't compare a **Button** with a **Label**, for instance
  - You can compare an **Object** with any object type



Express Yourself

- **Exercise 2**: Here are some CodePad exercises
  - Create three **Fraction** objects (**a**, **b**, **c**) : 1/2, 1/2, 1/3
  - Create two **Strings** (s1, s2), containing "Hi Mom"
  - What happens with the following comparisons:
    - **s1 == s2**
    - **s1 > s2**
    - **a == c**
    - **a > c**
    - **a == b**
    - **a != s1**

Shoot me a screen-shot.

The instanceof Operator

- A 7th relational operator works **only** with objects
  - Uses keyword **instanceof** instead of a symbol
  - **true** if **object** on left is instance of **class** on right
  - Also if object belongs to a subclass of class on right

```
Button b = new Button("Hi");
Menu m = new Menu("File");
boolean ans;

ans = b instanceof Bu
ans = b instanceof Co
ans = m instanceof Me
ans = m instanceof Co
```

Exercise 3: See if s1 is an instance of Fraction, String, Object. See if a is an instance of Number, Object.

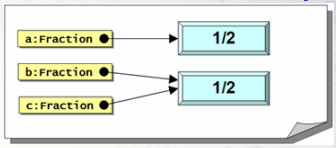
Simple Equality

- With objects, == measures **identity** or **simple equality**
  - Two variables refer to the **same** object or **String**
  - Here's an example with three **Fraction** variables

```
Fraction a = new Fraction(1, 2);
Fraction b = new Fraction(1, 2);
Fraction c = b;
```

Fractions in Memory

- In memory, the **Fractions** and variables look like this



- Here's what happens when we compare these:

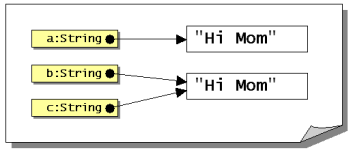
```
a == b; // false - different Fraction objects
b == c; // true - same Fraction
a == c; // false - different Fractions
```

Strings Comparisons

- **Strings** are objects; comparisons work the same way
- If you look at this code:

```
String a = "Hi Mom";
String b = "Hi Mom";
String c = b;
```

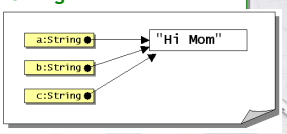
- You'd expect to see this in memory:



Strings in Memory

- With the same boolean expressions, though:

```
a == b; // true??? - Why?
b == c; // true - same String
a == c; // true???
```



- The reason the **Strings** are identical is because **Strings** are **immutable**, so memory really looks like this:

Value Equality

- Value equality: two objects have the **same contents**
  - This varies on a type-by-type basis
- Types that are comparable in this way will overload the **Object** method called **equals()**
- **Exercise 4**: Try this in CodePad:

```
String name = "Freddie".substring(0, 4);
name == "Fred";
name.equals("Fred");
```

Comparing Strings

- Methods for **comparing Strings**
  - **s == s2**
    - Compares the addresses in the variables s and s2
    - Returns true if s and s2 refer to the same string
  - **s.equals(s2), s.equalsIgnoreCase(s2)**
    - Returns true if two strings have the same contents
    - Lexical, not dictionary comparison
  - **s.compareTo(s2)**
    - Compares contents of **String** s and the **String** s2. Returns 0 if equal, positive if s is "greater" than S2, negative if less

Express Yourself

- **Exercise 5**: Add an **equals()** method to the **Fraction** class. This is a predicate method (that is, it will return **true** or **false**). It takes one **Fraction** as a parameter. It should return **true** if two objects have the same address, and it should also return **true** if two **Fraction**s have both the same numerator and same denominator.
- **Exercise 6**: Add a test program EX11A. Create three Fractions (1/2, 1/2, 1/3). Test your **equals()** method several different ways. Shoot a screen-shot.

What Card Was Picked?

- Run the **EX11B** class in project **ic11**, then open it.
  - A **GRect** table with two **GImage** cards face down
  - Want to flip over the card that was clicked
  - Use **getElementAt(x, y)** method to get a reference to the object that was clicked.
    - Save in a **Object** variable named **picked**

```
int x = e.getX();
int y = e.getY();
Object picked = getElementAt(x, y);
```

Simple If Actions

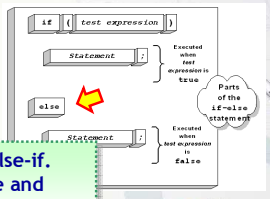
- You can compare an **Object** against **any** object type
  - Compare **picked** to **card1** using an **if** statement
    - If **true**, use **setImage()** to "flip" the card over

```
if (picked == card1)
   card1.setImage(getRandomCard());
```

  - Repeat for the **card2** variable
- **Exercise 7**: Run and make sure you can flip each card by clicking on it. Copy and paste your code.

A Small Problem

- Your program is **misleading** and **not very efficient**
  - If you pick **card1**, your code still checks **card2**
  - Not necessary because choice is **mutually exclusive**
- Java's **if** has an **else** portion that can be efficiently used for such **either/or** conditions



Exercise 8: Modify to use else-if. Snap a picture of your code and paste it into ic11.doc

Blocks and Indentation

- Most of the time, you need to carry out **several** actions inside the body of an **if** statement, not just one
- To do this, surround the body with braces (a "block")

```
if ( amountSold <= 35000)
{
    bonusPct = .035;
    bonusAmt = amountSold * bonusPct;
}
```

- Indent the code inside the block by one tab stop

Why Use Braces?

- Braces are not ***required*** for a single-line **if** statement
- Still a good idea, even when not required
  - Helps avoid mistakes when modifying code

```
if (amountSold <= 35000)
    bonusPct = .035;
else
    bonusPct = .075;
    bonusAmount += 100;
bonusAmount += amtSold * BonusPct;
```

The Phantom Semicolon

- Common mistake: a semicolon after the condition

```
if (filesAreAllBackedUp);
{
    formatTheHardDrive();
}
```

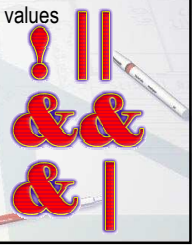
- Because of the semicolon, this fragment of code says:
  - 1. Check to see if files are backed up
  - 2. If they are, don't do anything (the **null** statement)
  - 3. Format the hard-drive regardless of the backup

The Cigar Party

- Log into your JavaBat account
  - In the Logic problems, locate Cigar Party
- **Exercise 9**: Spend 5 minutes using if and else to try and solve the Cigar Party problem shown here.
  - When squirrels get together for a party, they like to have cigars. A squirrel party is successful when the number of cigars is between 40 and 60, inclusive. Unless it is the weekend, in which case there is no upper bound on the number of cigars. Return true if the party with the given values is successful, or false otherwise.
- Snap a picture of your best run and place it in ic11.doc

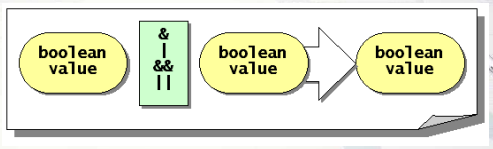
Logical Operators

- Many times, you need to test **several** values at once
  - Check if an input falls between 1 and 100, for instance
- Java's logical operators can to **combine** several tests
  - Logical operators work on **boolean** values
- One unary logical operator: NOT (**!**)
- Five binary logical operators
  - Two versions of AND (**&&**, **&**)
  - Two versions of OR (**||**, **|**)
  - One exclusive OR (**^**)



The Binary Logical Operators

- Two versions of **AND**, two versions of **OR**
  - Each requires two **boolean** operands
  - Each produces a **boolean** result



The AND Operators

- The symbol used by Java for **AND** is **&&** or **&**
- Corresponds to normal use of word "and"
  - **Both** conditions must be **true** for a **true** result
  - If either condition (or both) is **false**, result is **false**

```
if ( units >= 123 && GPA >= 1.5)
    youGraduate();
```

The OR Operators

- The symbol used by Java for **inclusive OR** is **||** or **|**
- Corresponds to natural understanding of OR
  - If **any** condition is **true**, the result is **true**
  - Both conditions must be **false** for a **false** result

```
if ( quiz > 75 || homework > 95 )
    finalGrade = 'B';
```

- The logical **exclusive OR** operator (**^**) is rarely used
  - If **exactly one** condition is **true**, the result is **true**

Express Yourself

- **Exercise 10**: Use the logical operators to complete the Cigar Party problem and snap a picture of your best run.

How Much to Evaluate?

- Sometimes, you "know" the answer to an expression without evaluating the entire thing.
  - Wheel of Fortune when the answer is obvious

Short-circuit Evaluation

- When using the `||` and `&&` operators, Java **stops** evaluating once it can determine the result
- This is called **short-circuit** or **lazy** evaluation
  - The expression is evaluated from left to right
  - Evaluation **stops** when the result is assured
- For `&&` stops when a `false` value is encountered
- For `||` stops when a `true` value is encountered
- `|` and `&` **evaluate both sides** regardless

Short-circuit Examples

- When should you use short-circuit and when should you use the "full-evaluation" operators?
  - Most of the time, you'll use the short-circuit operators
  - Here are some examples

```
if ( (a != 0) && ( b / a > 12 )) // OK
if ( (a != 0) & ( b/a > 12))     // NOT OK
if ( (a > 10) || (b++ > 7))      // Problem?
if ( (a > 10) | (b++ > 7))       // OK
```

Precedence

- Relational operators appear below arithmetic operators

```
isTrue = a > 3 + 5;  // OK, parens not required
```

- Logical operators appear below relational operators

```
isTrue = a > 3 || a < 1;   // parens not required
```

- Logical AND is higher than logical OR

```
isTrue = 10 < 15 || 5 > 8 && 3 > 5;
```